

Fonaments de Programació

Tutorial: Eines per a desenvolupadors

Aprendre Python com principiants pot ser una mica aclaparador. Per on començar? On podeu consultar informació? Com resoldre errors? Quin estil feu servir a l'hora de programar? Què feu un cop que heu après els conceptes bàsics? En aquest tutorial, hem compilat una llista d'alguns dels millors recursos de consulta i aprenentatge de Python per a principiants.

Com resoldre errors i excepcions

[Stack Overflow](#) és una plataforma molt popular, amb un format estil fòrum, on es poden obtenir respostes detallades, consells i trucs per solucionar qualsevol problema relacionat amb la programació. Us serà molt útil per trobar solucions a problemes o errors en el vostre codi. No obstant això, eviteu copiar codi del fòrum directament. El codi publicat a *Stack Overflow* és gratuït, sota l'atribut *Creative Commons* i amb llicència per compartir. Això vol dir que si feu servir codi de *Stack Overflow*, esteu obligats a donar crèdit a l'autor original i a fer que el vostre codi també estigui disponible de forma gratuïta. Enlloc de copiar codi, intenteu fer servir la informació que comparteixin els altres usuaris per aprendre els conceptes i aplicar-los mentre escriviu el vostre propi codi original. A més la plataforma ofereix una sèrie de funcionalitats que us seran de gran utilitat per aprofitar *Stack Overflow* al màxim:

- **Preguntes relacionades:** a la dreta de la pàgina trobareu un llistat de preguntes enllaçades (*Linked*) i relacionades (*Related*). Si la pregunta que heu trobat o la resposta no és satisfactòria, us aconsellem fer una ullada a aquests llistats, solen ser molt útils per a navegar i finalment trobar el fil que esteu buscant. A més les preguntes relacionades o *Related* també us poden servir per aprofundir en el tema que esteu explorant.
- **Tags:** si feu clic a una etiqueta o *tag* dels que apareixen sota una pregunta, arribareu a una pàgina que mostra totes les preguntes que tenen aquesta etiqueta. Us pot servir per buscar més informació sobre una mateixa categoria o tema.
- **Puntuació de les respostes:** fixeu-vos en que les respostes apareixen ordenades segons el seu nombre de vots o puntuació. Això us resultarà molt útil per diferenciar ràpidament les respostes més útils o fiables.

Abans de publicar qualsevol dubte a *Stack Overflow* busqueu i assegureu-vos que ningú ha formulat la mateixa pregunta abans. Molts de vosaltres us trobeu en una fase molt inicial d'aprenentatge dins de el món de la programació i és molt probable que els dubtes que us sorgeixin ja hagin estat tractats a la plataforma. Cercar abans de preguntar és una solució més ràpida i és una bona pràctica entre programadors. Si no trobeu la pregunta que busqueu i decidiu obrir un fil nou, tingueu en compte que les preguntes que són massa àmplies, poc clares, incompletes o basades principalment en opinions poden ser tancades per la comunitat fins que es millorin. Vegem com fer bon ús de *Stack Overflow* a través d'un exercici.

Si executeu el següent codi veureu que retorna un error a causa d'una sintaxi incorrecta: `SyntaxError: invalid syntax`.

```
In [11]:
```

```
while True print('Hola món')
```

```
File "<ipython-input-11-c090334ceed8>", line 1
    while True print('Hola món')
            ^
```

```
SyntaxError: invalid syntax
```

Com potser ja sabreu, Python és un llenguatge de programació que fa servir un intèrpret per a "traduir" el vostre codi al llenguatge màquina. Quan s'executa un *script* de Python, l'intèrpret avaluarà el vostre codi abans que realment l'executi. En cas que l'intèrpret trobi un error, aturarà el programa i us informará del tipus d'error i en quina part del codi es troba.

En l'exemple anterior podeu veure que l'intèrpret de Python ha retornat (ha advertit sobre) un error de sintaxi o `SyntaxError`. Això vol dir que l'intèrpret ha trobat un error en la forma en què es va escriure aquest *script* (sintaxi) i, com a conseqüència, ha aturat la seva execució immediatament.

En el missatge d'error, s'indica el tipus d'error que s'ha detectat, i veureu que el parser repeteix la línia infractora i mostra una petita "fletxa" que apunta al punt de la línia on s'ha detectat l'error. L'error és causat per (o almenys detectat en) la comanda que precedeix a la fletxa. En aquest cas, l'error es detecta en la funció `print`. Això es deu al fet que l'intèrpret va intentar analitzar la

precedeix a la lleixa. En aquest cas, l'error es detecta en la funció `print`. Això es deu al fet que l'interpret va intentar analitzar la funció `print` quan el bucle `while` encara no havia acabat. Recordeu que en Python, per convenció, l'ús de `while`, `for`, `if`, etc, s'ha d'acompanyar de `:`.

A més, fixeu-vos que el nom de l'arxiu i el número de línia s'imprimeixen perquè sàpigueu on buscar en el cas que l'entrada provingui d'un *script* amb múltiples línies o arxius.

Exercici 1

Executa el següent codi i indica quin tipus d'error has obtingut. A quin element del codi fa referència aquest error i a què creus que es deu?

In []:

```
4 + spam*3
```

Resposta: [Escriu aquí la teva resposta]

Com veiem en l'exercici anterior, més enllà dels errors de sintaxi, un codi també ens pot retornar altres tipus d'error. Això es deu al fet que de vegades una expressió és sintàcticament correcta però tot i així genera un error quan s'intenta executar. Aquests errors detectats durant l'execució s'anomenen excepcions i no sempre són crítics.

Els errors anteriors són bastant intuïtius a l'hora de reconèixer les seves causes i donar-los solució, però, altres vegades ens trobarem amb errors que no sabem què signifiquen exactament o que no sabem com resoldre. Què hem de fer en aquests casos? Aquí és quan és fonamental que el programador faci ús dels recursos *online* i el suport de comunitats de programadors com *Stack Overflow*. Vegem un exemple.

El següent codi ens retorna un error de tipus `TypeError`:

In []:

```
longitud = input('Indica la longitud en m:')
amplaria = input('Indica la amplària en m: ')
metres_quadrats = longitud * amplaria;
print (metres_quadrats)
```

Exercici 2

Busca l'error entre cometes en un motor de cerca (ex. Google) o en *Stack Overflow* i llegeix detingudament el primer resultat que obtinguis.

Què creus que significa aquest error i com podries solucionar-ho? En base als resultats obtinguts en la teva recerca, intenta reescriure el codi per resoldre el `TypeError`.

In []:

```
## Resposta: [Escriu aquí la teva resposta]
```

Llibries

Més enllà dels errors i les excepcions que ens puguem trobar, també és important que sapiguem accedir als recursos i a la documentació de les llibreries (de programari) que estem fent servir o que necessitem usar en el nostre codi. Una llibreria (o biblioteca) és una col·lecció d'arxius (anomenats mòduls) que contenen funcions per al seu ús per altres programes. També poden contenir valors de dades (per exemple, constants numèriques) i altres coses. La llibreria més usada en Python és la *Python Standard Library*, que està formada per un extens conjunt de mòduls que ve amb la instal·lació de Python per defecte. Alguns d'aquests mòduls són `random`, `math`, o `time` i contenen funcions com `math.cos(x)`, que retorna el cosinus d'un valor `x`, o `time.sleep(x)`, que pausa l'execució durant `x` segons. Però com podem saber què fa cada un d'aquests mòduls i funcions i com hem de fer-los servir? Tenim diverses opcions:

1- Podem accedir a la informació sobre les llibreries a través de la seva documentació *online*, com per exemple en el cas de la llibreria estàndard de Python, a través de la pàgina <https://docs.python.org/3/library/index.html>

2- Alternativament podem utilitzar la comanda `help(argument)` per conèixer el contingut d'un mòdul de la biblioteca que ens interessa. Per fer això, sempre tingueu en compte importar la llibreria o mòdul pertinent abans de fer la crida a la funció `help(argument)`. Recordeu també que haureu de passar un argument, si no es proporciona cap argument, el sistema d'ajuda

interactiva s'inicia a la consola de l'interpret i haureu d'introduir `quit` per sortir. Per exemple, per utilitzar la funció `help(argument)` per obtenir informació sobre la funció `print`, escriurem:

In [12]:

```
help(print)
```

Help on built-in function print in module builtins:

```
print(...)  
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)  
  
    Prints the values to a stream, or to sys.stdout by default.  
    Optional keyword arguments:  
    file: a file-like object (stream); defaults to the current sys.stdout.  
    sep:   string inserted between values, default a space.  
    end:   string appended after the last value, default a newline.  
    flush: whether to forcibly flush the stream.
```

Exercici 3

Donades les variables any, mes i dia, com generaries una data en el format iso estàndard? Quin mòdul de biblioteca estàndard podria servir-te? Quina funció seleccionaries d'aquest mòdul? Investiga els mòduls de la *Python Standard Library*, no dubtis a utilitzar un motor de cerca o *Stack Overflow* per aprofundir, i intenta escriure un programa que faci servir la funció.

In [1]:

```
any = 2016  
mes = 10  
dia = 22
```

```
## Resposta:
```

Guia d'estil

Una cosa que pot ser una mica complicada quan ens iniciem en un nou llenguatge de programació és llegir i entendre el codi que escriuen altres persones. Per evitar confusions i facilitar la llegibilitat entre programadors de diferents llocs, cultures i formacions, és una bona pràctica seguir uns estàndards d'estil. [PEP 8](#), de vegades escrit PEP8 o PEP8, és un document que proporciona les pautes i les millors pràctiques sobre com escriure codi Python. Va ser escrit en 2001 per Guido van Rossum, Barry Warsaw i Nick Coghlan. L'enfocament principal de PEP 8 és millorar la llegibilitat i la coherència del codi Python.

La bona notícia és que no haureu de memoritzar tot el contingut de la guia. Hi ha molts plugins i eines *online* que us serviran per verificar si el vostre codi és compatible amb la guia d'estil PEP8 i fins i tot corregiran alguns aspectes per vosaltres, per exemple, espais en blanc finals. Però no cregueu que el vostre codi és compatible amb PEP8 si un plugin no us dóna cap revisió de resultats. Per exemple, aquests plugins no poden comprovar si les vostres variables tenen noms significatius o si els noms són coherents. Per això us recomanem llegir la Guia d'estil PEP8 de tant en tant, per assegurar-vos que no esteu passant per alt cap aspecte relacionat amb l'estil.

Exercici 4

Per exemple, `autopep8` es pot utilitzar en Jupyter per reformatar/embellir el codi en la cel·la de codi d'un quadern. Només us caldrà instal·lar el paquet corresponent:

In [14]:

```
!pip install autopep8
```

```
/bin/sh: pip: command not found
```